

Project Report

Due Date: 29/08/2026

Project ID & Title: #3 - Freelance Marketplace Ecosystem (Mini-Upwork)

A. Project Identity

- **Team Name:** Rum
- **Team Members:**
 - Lý Nguyễn Thành Đạt (n24dece060@student.ptithcm.edu.vn)
 - Hồ Nguyên Sâm (n24dece089@student.ptithcm.edu.vn)
 - Trần Quang Huy (n24dece073@student.ptithcm.edu.vn)
- **Project Title:** Upwork - Freelance Marketplace Contract and Payment Management Database

1. Introduction and Project Scope

1.1 System Objective

1.1.1 Background and Problem Statement

Upwork is an academic database system developed for Upwork.com. It connects clients who need work completed with freelancers who provide professional services. The system supports the complete freelance lifecycle: account registration, job posting and discovery, proposal submission, contract formation, work tracking, escrow funding, hourly invoicing, payment settlement, and two-way review after contract completion.

The central database problem is not merely storing users, jobs, and contracts. These objects participate in dependent workflows whose integrity must be preserved across status changes and financial events. The database must prevent duplicate proposals, self-bidding, contracts created from unaccepted proposals, mismatched fixed-price and hourly data, overlapping time entries, cross-contract payment references, unauthorized reviews, and destructive modification of completed financial history.

Fixed-price contracts divide agreed work into milestones. A milestone is funded before protected work begins, submitted by the freelancer, reviewed by the client, and released through escrow when the acceptance conditions are satisfied [3], [7]. Hourly contracts collect eligible time entries into a weekly invoice and follow a weekly review and billing cycle [4], [5]. Successful transactions remain auditable, and corrections are represented by compensating transactions rather than overwriting financial history.

1.1.2 Domain Survey and Analysis

The team surveyed official Upwork Help Center and legal documentation and identified five major business processes:

1. **Job posting and proposal handling:** A client publishes a job, receives proposals, and may shortlist, decline, or hire a freelancer [1], [2].
2. **Fixed-price work:** The contract is organized into milestones. Each milestone records a deliverable, due date, amount, funding state, submission state, and release state [3].
3. **Hourly work:** The freelancer records time entries. Eligible entries are grouped into a weekly invoice for client review and payment [4], [5].

4. **Escrow and settlement:** Fixed-price funds are deposited, held, released, refunded, or reversed through controlled financial events [7].
5. **Post-contract feedback:** After a contract is closed, the parties may leave a 1-to-5 rating and optional comment [6].

The survey shows that the conceptual model must prioritize lifecycle consistency, subtype separation, source-to-payment traceability, party validation, immutable financial history, and correct rating aggregation. The Mini-Upwork model deliberately simplifies real-world banking, regulatory, dispute, tax, and identity-verification processes while preserving the essential marketplace rules.

1.1.3 Project Objectives

- Represent client and freelancer roles as specializations of a common user account.
- Store freelancer skills, job categories, and job skill requirements.
- Support fixed-price and hourly job posts without subtype ambiguity.
- Control proposal, acceptance, and contract-formation lifecycles.
- Manage milestones and escrow funding for fixed-price contracts.
- Manage time entries and weekly invoices for hourly contracts.
- Record auditable payment transactions with validated payer, payee, contract, and billing source.
- Support two-way contract reviews and derived rating summaries.
- Keep identifiers and timestamps ready for a later distributed deployment design without prematurely fixing a physical architecture.

1.1.4 Project Scope

In scope

- User accounts; client and freelancer profiles; dual-role users.
- Skills, categories, hierarchical subcategories, and job skill requirements.
- Fixed-price and hourly jobs, proposals, and contract formation.
- Fixed-price milestones, escrow funding, approval, release, refund, and reversal traceability.
- Hourly time entries, weekly invoices, and weekly payment settlement.
- Payment transaction history with payer/payee and source validation.
- Two-way reviews, rating summaries, status transitions, and deletion restrictions.
- Conceptual identifiers and UTC timestamps that remain compatible with later distribution decisions.

Out of project scope

- Agency accounts, freelancer teams, payroll employment, and enterprise procurement.
- Real banking credentials, external payment-gateway integration, tax, foreign-exchange, and regulatory compliance.
- Messaging, video calls, file storage, identity-document verification, and dispute arbitration.
- AI job recommendations, mobile application development, and actual keyboard/mouse/screenshot monitoring.
- A full reproduction of Upwork's proprietary production infrastructure.

1.1.5 Stakeholders and User Requirements

Table 1.1. Stakeholder requirements

ID	Stakeholder	User requirement
UR-C01	Client	Create and maintain a client profile.
UR-C02	Client	Post a fixed-price or hourly job with a category and required skills.
UR-C03	Client	View, shortlist, decline, or accept freelancer proposals.
UR-C04	Client	Create a contract from one accepted proposal.
UR-C05	Client	Fund, review, approve, release, or refund fixed-price milestones.
UR-C06	Client	Review hourly invoices and payment status.
UR-C07	Client	Review the freelancer after contract completion.
UR-F01	Freelancer	Maintain a freelancer profile and professional skills.
UR-F02	Freelancer	Search open jobs by category, skill, contract type, and budget/rate.
UR-F03	Freelancer	Submit, update, withdraw, or resubmit one proposal per job.
UR-F04	Freelancer	Submit milestone work or record hourly time entries.
UR-F05	Freelancer	Monitor escrow, invoice, and payment status.
UR-F06	Freelancer	Review the client after contract completion.
UR-A01	Database administrator	Protect credentials and financial data with constraints, transactions, backups, auditing, and least-privilege roles.

1.1.6 Functional Requirements

Table 1.2. Functional requirements and traceability

ID	Functional requirement	Related rules
FR-01	Create a user with a unique email address and one or both marketplace roles.	BR-01-BR-02
FR-02	Manage freelancer skills and proficiency levels.	BR-03
FR-03	Create a typed job with one category and required skills.	BR-04-BR-10
FR-04	Search only open jobs by category, skill, type, budget, or hourly rate.	BR-05-BR-10
FR-05	Maintain at most one proposal lifecycle per freelancer-job	BR-10-BR-14

	pair.	
FR-06	Create one type-compatible contract from one accepted proposal.	BR-15-BR-20
FR-07	Manage fixed-price milestones and their controlled lifecycle.	BR-21-BR-24
FR-08	Record valid, non-overlapping time entries for active hourly contracts.	BR-25-BR-27
FR-09	Generate one weekly invoice per contract and billing period.	BR-28-BR-29
FR-10	Fund escrow and record payment transactions against exactly one valid billing source.	BR-30-BR-37
FR-11	Allow the two contract parties to review each other after completion.	BR-38-BR-41
FR-12	Calculate a rating summary from valid received reviews.	BR-42
FR-13	Preserve completed contract and financial history without physical deletion or destructive edits.	BR-36, BR-43

1.1.7 Non-Functional Requirements

Table 1.3. Non-functional requirements

ID	Quality	Requirement
NFR-01	Integrity	Proposal, contract, work, escrow, payment, and review operations shall be atomic and preserve all lifecycle and cross-contract constraints.
NFR-02	Performance	With 10,000 jobs and 50,000 proposals, indexed job-search queries should complete within two seconds on the documented test environment.
NFR-03	Security	Passwords shall be stored as salted hashes; database privileges shall follow least privilege.
NFR-04	Auditability	Successful payments and completed contracts shall not be physically deleted or destructively modified by application users.
NFR-05	Recoverability	A tested backup-and-restore procedure shall recover committed rows to the stated recovery point.
NFR-06	Maintainability	Identifiers shall use meaningful lowercase snake_case names and controlled domain values.
NFR-07	Privacy	Authentication and payment-sensitive data shall not be exposed by public-profile queries.

NFR-08	Distribution readiness	Identifiers shall be globally unique, timestamps stored in UTC, and currency represented by an ISO-style code so later distribution does not require key redesign.
--------	------------------------	--

1.2 Business Rules and Constraints

Table 1.4. Business rules and planned enforcement

ID	Business rule	Planned enforcement
BR-01	Each user has an immutable user_id and a unique case-insensitive email address.	PK, UNIQUE
BR-02	Every marketplace user is a CLIENT, a FREELANCER, or both. The specialization is total and overlapping.	EER specialization
BR-03	Each freelancer-skill pair occurs once; proficiency_level is a star from 1 to 5.	Composite PK, CHECK
BR-04	Each job is posted by exactly one client; a client may post zero or many jobs.	FK, cardinality
BR-05	Each job belongs to exactly one category; a category may classify zero or many jobs.	FK, cardinality
BR-06	Each job is exactly one subtype: FIXED_PRICE_JOB or HOURLY_JOB. The specialization is total and disjoint.	EER specialization
BR-07	Fixed-price budgets and hourly rates are positive; rate_min does not exceed rate_max.	CHECK
BR-08	Category names and skill names are unique under case-insensitive comparison.	UNIQUE
BR-09	A job has at least one required skill before its status becomes OPEN.	Deferred validation
BR-10	Job status follows DRAFT -> OPEN -> CLOSED or CANCELLED; only OPEN jobs accept proposals.	Transition trigger
BR-11	At most one proposal record exists for each freelancer-job pair; withdrawal and resubmission update the same lifecycle.	UNIQUE
BR-12	A user cannot submit a proposal to a job posted by the same underlying user account.	Trigger
BR-13	A proposal's offered amount is positive and interpreted according to the originating job subtype.	CHECK, subtype validation
BR-14	Proposal status follows SUBMITTED ->	Transition

	SHORTLISTED/ACCEPTED/REJECTED/WITHDRAWN; only withdrawn proposals may be resubmitted.	trigger
BR-15	Only an ACCEPTED proposal may create a contract.	Transaction or trigger
BR-16	Each proposal creates at most one contract.	UNIQUE
BR-17	Within this Mini-Upwork scope, each job has at most one accepted proposal and one resulting contract.	Partial UNIQUE or trigger
BR-18	Every contract is exactly one subtype: FIXED_PRICE_CONTRACT or HOURLY_CONTRACT. The specialization is total and disjoint.	EER specialization
BR-19	The contract subtype matches the subtype of the proposal's originating job.	Trigger or transaction
BR-20	A contract requires start_date; end_date is null while active or not earlier than start_date. Status follows DRAFT -> ACTIVE/PAUSED -> COMPLETED or TERMINATED.	CHECK, transition trigger
BR-21	A fixed-price contract has at least one milestone before it becomes ACTIVE.	Deferred validation
BR-22	Milestone sequence numbers are unique within a contract and every milestone amount is positive.	UNIQUE, CHECK
BR-23	The sum of planned milestone amounts equals the agreed fixed-price amount before activation.	Deferred validation
BR-24	Milestone status follows PLANNED -> FUNDED -> SUBMITTED -> APPROVED -> RELEASED, with controlled CANCELLED or REFUNDED alternatives.	Transition trigger
BR-25	Hourly rate and weekly hour limit are positive.	CHECK
BR-26	Time entries are recorded only for an ACTIVE hourly contract.	Trigger
BR-27	Each time entry satisfies start_at < end_at, does not exceed the weekly limit, and does not overlap another entry for the same contract.	CHECK, overlap trigger
BR-28	A weekly invoice is unique per hourly contract and billing period; period_start < period_end and periods do not overlap.	UNIQUE, CHECK
BR-29	An invoice summarizes one or more eligible entries; each time entry belongs to at most one invoice. Totals equal eligible hours multiplied by the contract rate.	FK, transaction
BR-30	A milestone has at most one active escrow-funding record;	UNIQUE,

	funded_amount is positive and equals the funded milestone amount.	CHECK
BR-31	Escrow status follows PENDING -> FUNDED -> RELEASED or REFUNDED; currency_code is required and consistent with related payments.	Transition trigger
BR-32	Every payment amount is positive, belongs to exactly one contract, and records one currency_code.	FK, CHECK
BR-33	A payment references ESCROW_FUNDING XOR WEEKLY_INVOICE: exactly one source is present.	XOR CHECK
BR-34	The referenced escrow funding or invoice belongs to the same contract as the payment.	Trigger
BR-35	payer_id and payee_id are distinct contract parties and match the direction required by transaction_type.	Trigger
BR-36	A successful payment is immutable; a correction is a compensating transaction linked through original_payment_id.	RBAC, trigger
BR-37	Escrow release requires an APPROVED, FUNDED milestone; an hourly charge requires an approved weekly invoice.	Trigger or transaction
BR-38	A review is created only after the related contract is COMPLETED.	Trigger
BR-39	Reviewer and reviewee are the two opposite parties of the contract.	Trigger
BR-40	Each party submits at most one review per contract; one contract therefore has at most two reviews.	UNIQUE
BR-41	Rating is an integer from 1 to 5 and a user cannot review themselves.	CHECK
BR-42	Each user has at most one rating summary; count, sum, and average equal valid reviews received by that user.	PK/FK, view or trigger
BR-43	Referenced users, jobs, contracts, invoices, escrow records, payments, and reviews are closed or deactivated by status instead of physical deletion.	RESTRICT FK, RBAC

2. Database Design

2.1 Conceptual Model - ER/EER Diagram

2.1.1 Modeling Notation

The conceptual model uses UML class-diagram conventions from UML 2.5.1 [10]. Entity boxes show conceptual identifiers and major attributes. Association labels state the relationship and end multiplicities. Hollow triangular arrows indicate specialization/generalization. PK, FK, and UQ denote conceptual key, reference, and uniqueness constraints; [0..1] marks an optional reference.

Table 2.1. EER notation legend

Symbol	Meaning
1	Exactly one
0..1	Optional; at most one
0..*	Zero or many
1..*	One or many
Total	Every supertype instance belongs to at least one subtype
Disjoint	A supertype instance belongs to only one subtype
Overlapping	A supertype instance may belong to more than one subtype
XOR	Exactly one of the alternative relationships is present

Specialization constraints: USER -> CLIENT/FREELANCER is total and overlapping.
 JOB -> FIXED_PRICE_JOB/HOURLY_JOB and CONTRACT ->
 FIXED_PRICE_CONTRACT/HOURLY_CONTRACT are total and disjoint.
 PAYMENT_TRANSACTION references exactly one billing source: ESCROW_FUNDING
 XOR WEEKLY_INVOICE.

2.1.2 Comprehensive EER Overview

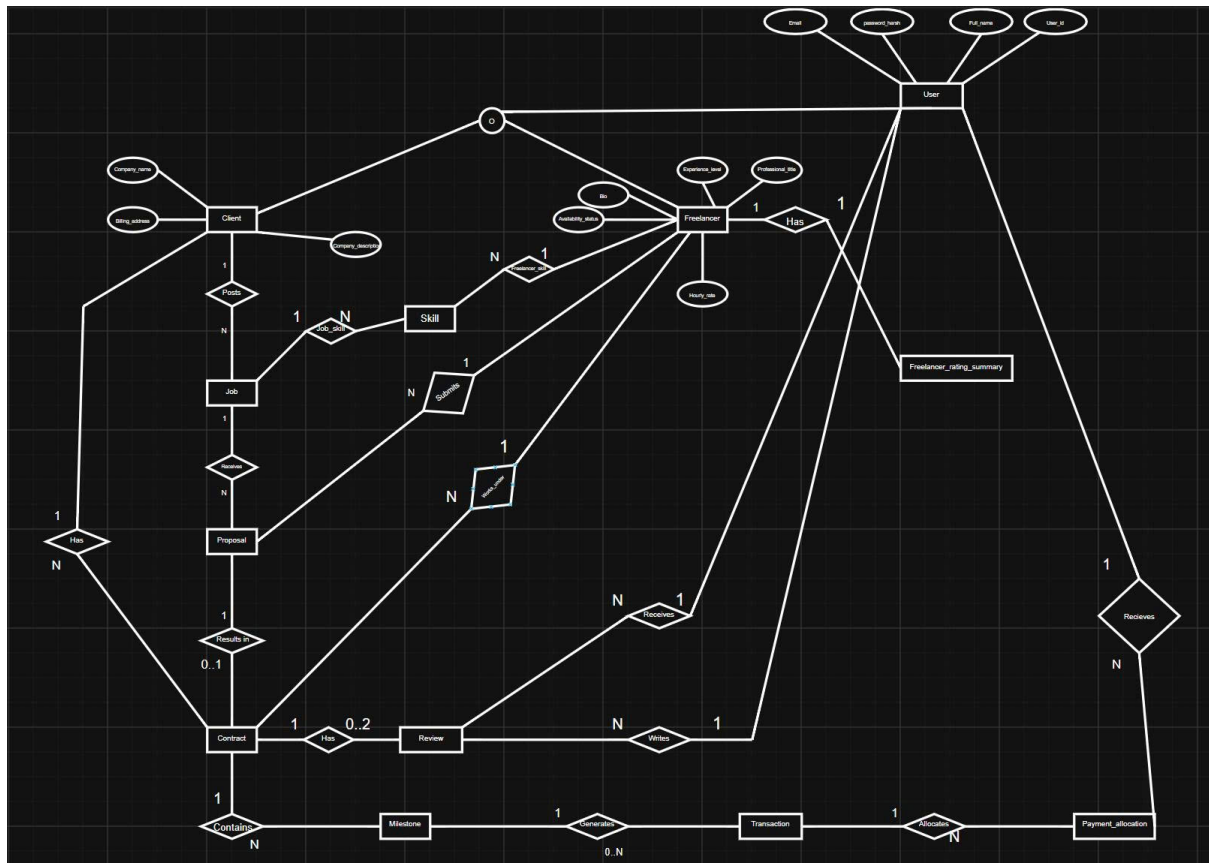


Figure 2.1. Comprehensive EER overview - all entities and relationships

The overview is the authoritative conceptual model. Figures 2.2 and 2.3 enlarge the same model into readable attribute-level views; they do not define separate databases.

2.1.3 Marketplace and Proposal Detail

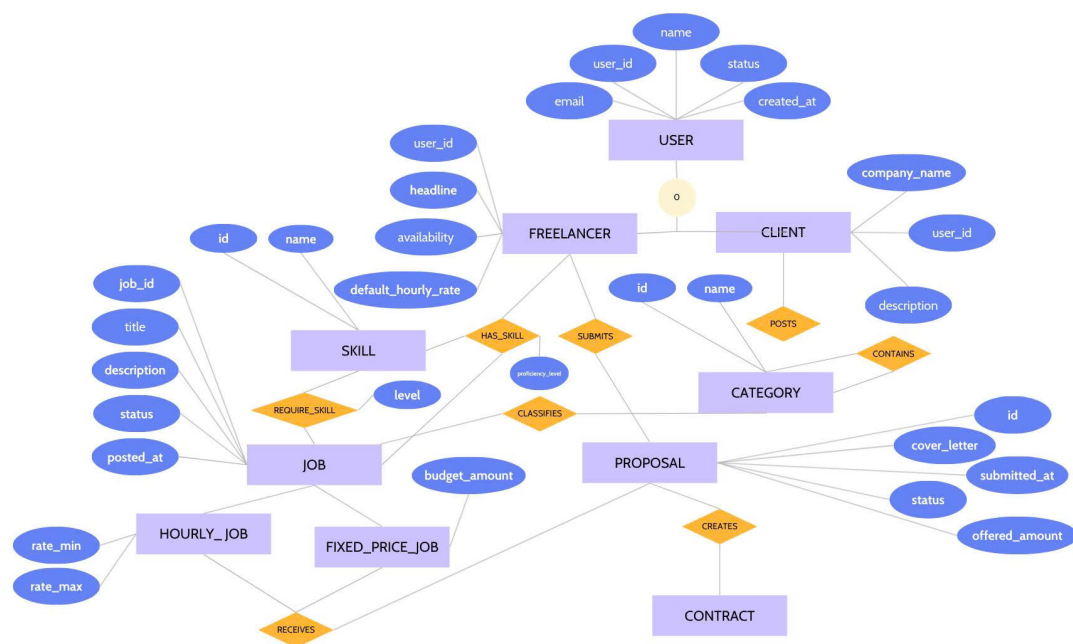


Figure 2.2. Detailed EER view - marketplace, skills, jobs, and proposals

2.1.4 Contract, Work, Escrow, Payment, and Review Detail

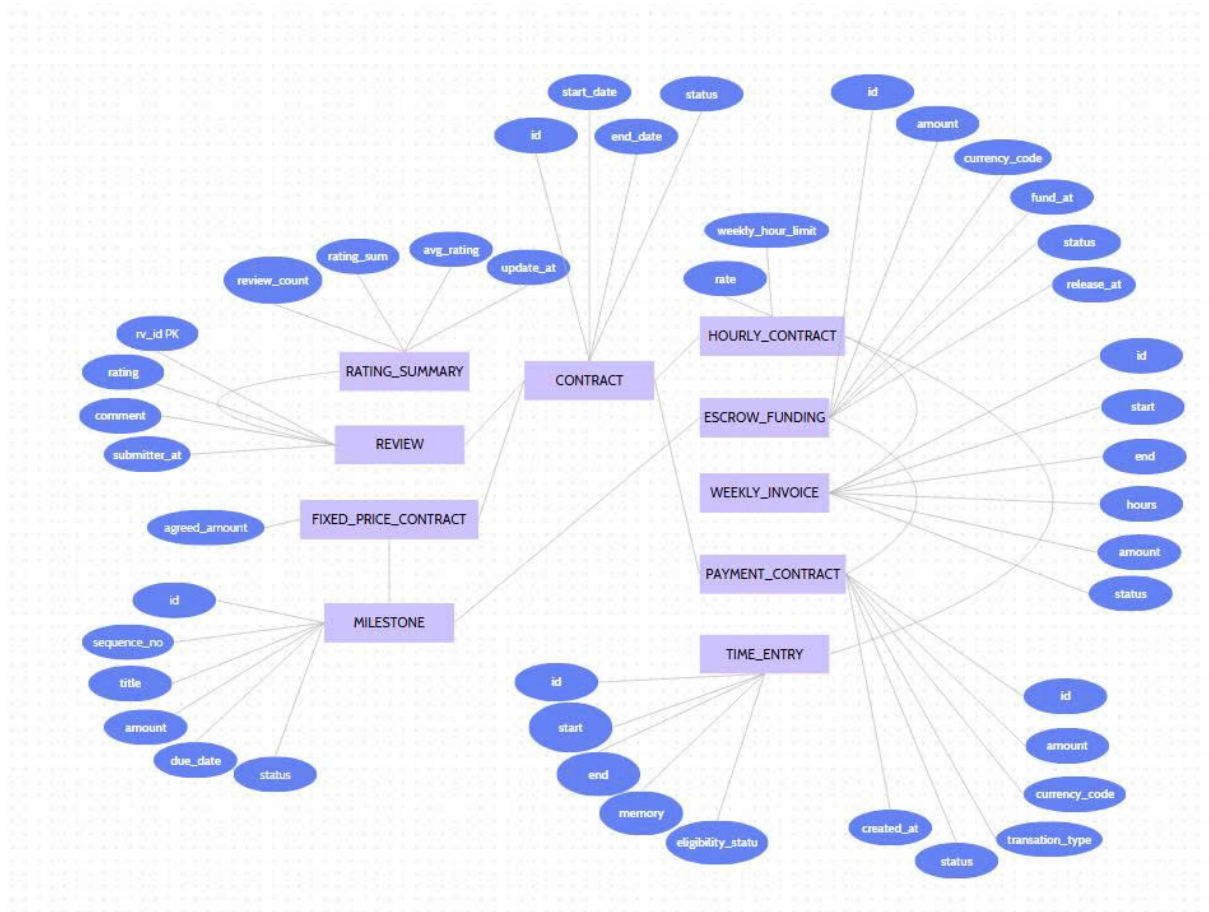


Figure 2.3. Detailed EER view - contract execution, settlement, and feedback

2.2 Entity Catalog

Table 2.2. Entity catalog

Entity	Type	Important attributes
USER	Supertype	user_id; email; password_hash; full_name; status; created_at
CLIENT	Subtype	user_id; company_name; description
FREELANCER	Subtype	user_id; headline; overview; default_hourly_rate; availability
CATEGORY	Strong/recursive	category_id; parent_category_id;

		category_name
SKILL	Strong	skill_id; skill_name
FREELANCER_SKILL	Associative	freelancer_id; skill_id; proficiency_level
JOB	Supertype	job_id; client_id; category_id; title; description; status; posted_at
FIXED_PRICE_JOB	Subtype	job_id; budget_amount
HOURLY_JOB	Subtype	job_id; rate_min; rate_max
JOB_SKILL	Associative	job_id; skill_id; required_level
PROPOSAL	Event	proposal_id; job_id; freelancer_id; cover_letter; offered_amount; status; submitted_at
CONTRACT	Supertype	contract_id; proposal_id; start_date; end_date; status
FIXED_PRICE_CONTRACT	Subtype	contract_id; agreed_amount
HOURLY_CONTRACT	Subtype	contract_id; hourly_rate; weekly_hour_limit
MILESTONE	Dependent	milestone_id; contract_id; sequence_no; title; amount; due_date; status
TIME_ENTRY	Dependent/event	time_entry_id; contract_id; invoice_id [optional]; start_at; end_at; memo; eligibility_status
WEEKLY_INVOICE	Aggregate/document	invoice_id; contract_id; period_start; period_end; total_hours; total_amount; status
ESCROW_FUNDING	Financial event	funding_id; milestone_id; funded_amount; currency_code; status; funded_at; released_at
PAYMENT_TRANSACTION	Transaction	payment_id; contract_id; escrow_funding_id [optional]; invoice_id [optional]; payer_id; payee_id; original_payment_id [optional]; amount; currency_code; transaction_type; status; created_at

REVIEW	Associative/event	review_id; contract_id; reviewer_id; reviewee_id; rating; comment; submitted_at
RATING_SUMMARY	Derived aggregate	user_id; review_count; rating_sum; average_rating; updated_at

2.3 Specialization and Generalization Constraints

Table 2.3. Specialization constraints

Supertype	Subtypes	Completeness	Disjointness	Interpretation
USER	CLIENT, FREELANCER	Total	Overlapping	Every marketplace user has at least one role; a dual-role user may have both.
JOB	FIXED_PRICE_JOB, HOURLY_JOB	Total	Disjoint	Every job has exactly one pricing type.
CONTRACT	FIXED_PRICE_CONTRACT, HOURLY_CONTRACT	Total	Disjoint	Every contract has exactly one execution/billing type matching its job.

2.4 Relationships and Cardinalities

Table 2.4. Relationship cardinalities

Relationship	Multiplicity at A	Multiplicity at B	Meaning
CLIENT posts JOB	Client: 0..* jobs	Job: exactly 1 client	Every job has one owner.
CATEGORY contains CATEGORY	Parent: 0..* children	Child: 0..1 parent	Supports a category hierarchy.
CATEGORY classifies JOB	Category: 0..* jobs	Job: exactly 1 category	Every job is classified.

FREELANCER has SKILL	Freelancer: 0..* skills	Skill: 0..* freelancers	Resolved by FREELANCER_SKILL.
JOB requires SKILL	Job: 0..*; 1..* before OPEN	Skill: 0..* jobs	Resolved by JOB_SKILL.
FREELANCER submits PROPOSAL	Freelancer: 0..* proposals	Proposal: exactly 1 freelancer	One lifecycle per freelancer-job pair.
JOB receives PROPOSAL	Job: 0..* proposals	Proposal: exactly 1 job	Only OPEN jobs receive proposals.
PROPOSAL creates CONTRACT	Proposal: 0..1 contract	Contract: exactly 1 proposal	Only an accepted proposal creates a contract.
FIXED_PRICE_CONTRACT contains MILESTONE	Contract: 0..*; 1..* before ACTIVE	Milestone: exactly 1 fixed contract	Hourly contracts cannot own milestones.
HOURLY_CONTRACT records TIME_ENTRY	Contract: 0..* entries	Entry: exactly 1 hourly contract	Entries exist only while active.
HOURLY_CONTRACT has WEEKLY_INVOICE	Contract: 0..* invoices	Invoice: exactly 1 hourly contract	One invoice per billing period.
WEEKLY_INVOICE summarizes TIME_ENTRY	Invoice: 1..* entries	Entry: 0..1 invoice	Unbilled eligible entries remain optional until invoiced.
MILESTONE has ESCROW_FUNDING	Milestone: 0..1 active funding	Funding: exactly 1 milestone	Funding exists only for fixed-price work.
CONTRACT owns PAYMENT_TRANSACTION	Contract: 0..* payments	Payment: exactly 1 contract	Every financial event is contract-traceable.
ESCROW_FUNDING sources PAYMENT	Funding: 0..* transactions	Payment: source A, optional	Deposit, release, refund, and reversal history may be retained.
WEEKLY_INVOICE sources PAYMENT	Invoice: 0..* transactions	Payment: source B, optional	Hourly charges, payouts, and reversals are retained.

USER is payer/payee	User: 0..* transactions	Payment: exactly 1 payer and 1 payee	Parties must be the contract client and freelancer.
CONTRACT permits REVIEW	Contract: 0..2 reviews	Review: exactly 1 contract	One review per party after completion.
USER writes/receives REVIEW	User: 0..* reviews	Review: 1 reviewer and 1 reviewee	Both users must be opposite contract parties.
USER has RATING_SUMMARY	User: 0..1 summary	Summary: exactly 1 user	Derived after at least one valid received review.

Exclusive-or constraint: PAYMENT_TRANSACTION.escrow_funding_id and PAYMENT_TRANSACTION.invoice_id are mutually exclusive and collectively mandatory: exactly one is non-null. The selected source must belong to the same contract as the payment.

2.5 Lifecycle and State Constraints

Table 2.5. Controlled lifecycle transitions

Entity	From	Allowed next state(s)	Guard
JOB	DRAFT	OPEN, CANCELLED	OPEN only after category and at least one required skill exist.
JOB	OPEN	CLOSED, CANCELLED	Only OPEN accepts proposals.
PROPOSAL	SUBMITTED	SHORTLISTED, ACCEPTED, REJECTED, WITHDRAWN	Only ACCEPTED creates a contract.
CONTRACT	DRAFT	ACTIVE, TERMINATED	Subtype must match job; required work structure must exist.
CONTRACT	ACTIVE	PAUSED, COMPLETED, TERMINATED	Completed contracts permit reviews.

MILESTONE	PLANNED	FUNDED, CANCELLED	Funding amount must be valid.
MILESTONE	FUNDED	SUBMITTED, REFUNDED	Work may be submitted only when funded.
MILESTONE	SUBMITTED	APPROVED, FUNDED	Return to FUNDED represents a change request.
MILESTONE	APPROVED	RELEASED	Release creates an immutable financial transaction.
ESCROW_FUNDING	PENDING	FUNDED, CANCELLED	Funds are not releasable while pending.
ESCROW_FUNDING	FUNDED	RELEASED, REFUNDED	Terminal outcomes preserve history.
WEEKLY_INVOICE	DRAFT	ISSUED	Totals are calculated from eligible entries.
WEEKLY_INVOICE	ISSUED	APPROVED, DISPUTED	Only approved invoices are payable in scope.
PAYMENT_TRANSACTION	PENDING	SUCCEEDED, FAILED	SUCCEEDED rows are immutable.

2.6 End-to-End Validation Scenarios

Scenario 1 - Fixed-price contract

A client posts a fixed-price job. A freelancer submits a proposal and the client accepts it. The system creates a fixed-price contract with milestones. The client funds a milestone, the freelancer submits work, the client approves it, and escrow is released. Expected result: no time entry or weekly invoice belongs to the contract; payments reference escrow funding only; payer/payee and contract match.

Scenario 2 - Hourly contract

A client posts an hourly job and accepts a compatible proposal. The freelancer records non-overlapping eligible time. A weekly invoice summarizes those entries. Expected result: no

milestone or escrow funding belongs to the contract; the invoice amount equals eligible hours multiplied by the hourly rate; payment references the invoice only.

Scenario 3 - Dual-role user

One USER has both CLIENT and FREELANCER profiles. The user may post a job and bid on other clients' jobs. Expected result: a proposal to the user's own job is rejected by the shared user_id rule.

Scenario 4 - Duplicate or invalid proposal

A freelancer submits a second proposal record to the same job, or submits to a non-OPEN job. Expected result: the unique pair and job-status rules reject the operation; resubmission must reuse the original proposal lifecycle.

Scenario 5 - Invalid payment source

An application supplies both escrow_funding_id and invoice_id, supplies neither, or references a source from another contract. Expected result: XOR and same-contract checks reject the transaction.

Scenario 6 - Invalid payment parties or correction

A non-party is recorded as payer/payee, or a successful transaction is directly edited. Expected result: party validation rejects the first operation; the second requires a new compensating transaction linked to the original.

Scenario 7 - Invalid review

A non-party reviews a contract, a review is submitted before completion, or one party submits a second review. Expected result: completion, party, and unique contract-reviewer constraints reject the review.

2.7 Requirement-to-Model Traceability

Table 2.6. Requirements-to-model traceability matrix

Capability	Requirements and rules	EER elements
User roles and profiles	FR-01-FR-02; BR-01-BR-03	USER, CLIENT, FREELANCER, FREELANCER_SKILL
Jobs and discovery	FR-03-FR-04; BR-04-BR-10	JOB specializations, CATEGORY, SKILL, JOB_SKILL
Proposals and contracts	FR-05-FR-06; BR-11-BR-20	PROPOSAL, CONTRACT specializations
Fixed-price work	FR-07; BR-21-BR-24	MILESTONE
Hourly work	FR-08-FR-09; BR-25-BR-29	TIME_ENTRY, WEEKLY_INVOICE
Escrow and payments	FR-10, FR-13; BR-30-BR-37, BR-43	ESCROW_FUNDING, PAYMENT_TRANSACTION

Reviews and ratings	FR-11-FR-12; BR-38-BR-42	REVIEW, RATING_SUMMARY
---------------------	--------------------------	---------------------------

Appendix A. Team Contribution Report

Table A.1. Member responsibilities and contribution

Member	Primary responsibility	Evidence/output	Share
Lý Nguyễn Thành Đạt	USER, CLIENT, FREELANCER, JOB, PROPOSAL; integration and report editing	Requirements; entity catalog; BR-01-BR-20; final consistency review	34%
Hồ Nguyên Sâm	EER structure, specialization, cardinalities, diagrams	Figures 2.1-2.3; relationship table; notation and traceability review	33%
Trần Quang Huy	CONTRACT, work tracking, escrow, payments, reviews	BR-21-BR-43; lifecycle table; validation scenarios	33%

Appendix B. Weekly Progress Log

Table B.1. Phase 1 weekly progress

Week	Activities	Participants	Output
Week 3	Survey official workflows; define problem, objectives, scope, stakeholders, UR/FR/NFR; draft entities and rules.	Đạt, Sâm, Huy	Requirements baseline and initial entity list
Week 4	Finalize business rules; build EER overview/detail diagrams; validate cardinalities, escrow/payment logic, scenarios, references, and report format.	Đạt, Sâm, Huy	Submission-ready Phase 1 report

References

- [1] Upwork Help Center, 'Review job proposals.' Accessed 29 August 2026.
<https://support.upwork.com/hc/en-us/articles/18010402882195--Review-job-proposals>

- [2] Upwork Help Center, 'How to send an offer from a job post.' Accessed 29 August 2026.
<https://support.upwork.com/hc/en-us/articles/211063618-How-to-send-an-offer-from-a-job-post>
- [3] Upwork Help Center, 'How payments for milestones and fixed-price contracts work.' Accessed 29 August 2026. <https://support.upwork.com/hc/en-us/articles/211063718-How-payments-for-milestones-and-fixed-price-contracts-work>
- [4] Upwork Help Center, 'How to manage the weekly billing cycle.' Accessed 29 August 2026.
<https://support.upwork.com/hc/en-us/articles/211063698-How-to-manage-the-weekly-billing-cycle>
- [5] Upwork Help Center, 'How hourly contracts work.' Accessed 29 August 2026.
<https://support.upwork.com/hc/en-us/articles/18926255847443--How-hourly-contracts-work>
- [6] Upwork Help Center, 'How to leave an end-of-contract review for your freelancer.' Accessed 29 August 2026. <https://support.upwork.com/hc/en-us/articles/211062188-How-to-leave-an-end-of-contract-review-for-your-freelancer>
- [7] Upwork, 'User Agreement and Escrow Instructions,' Upwork Legal Center. Accessed 29 August 2026.
<https://www.upwork.com/legal>
- [8] Course Project Topic List, 'Project #3 - Freelance Marketplace Ecosystem (Mini-Upwork),' academic handout, 2026-2027.
- [9] Milestone Project Plan and Evaluation, academic handout, 2026-2027.
- [10] Object Management Group, 'Unified Modeling Language (UML), Version 2.5.1,' December 2017.
<https://www.omg.org/spec/UML/2.5.1>